Project Summary

What We Built

A services company runs many client engagements, and each client has its own coding standards, deploy approvals, security & licensing rules, and approved tools. Developers who rotate between clients routinely mix up the rules - and the same question can have opposite correct answers depending on the client. Guessing wrong ships a licensing, security, or compliance violation.

We built the Multi-Client Standards Concierge: a conversational assistant where a developer asks a plain-English question and gets the right standard for the client they are on - routed automatically through client -> team -> topic - with an identity layer that limits each developer to the client they are authorized for. All data is synthetic.

The Problem

- Same question, different right answer. "Can I use a GPL library?" is *prohibited* for Banking but *allowed with architect sign-off* for Automotive.
- Rules bleed across clients. Yesterday's client's policy leaks into today's work.
- No single source of truth. Standards live in scattered docs no one re-reads.
- Access matters. A developer should not be able to read another client's standards.

How It Works

A Streamlit chat frontend talks to a neuro-san multi-agent backend.

The Agentic System

The network has three genuine routing levels plus four deterministic coded tools:

Standards_Concierge (front desk) - receives every message, establishes the developer's identity (via EmployeeDirectory) and current client (via ClientResolver), enforces authorization, then routes the question.

Client_Router (L1) -> Client_<X> (L2) -> <X>_Team_<Y> (L3) - three real AAOSA routing levels. Team depth is uneven on purpose (Banking A-D, Automotive A-C, ... Telecom A) so routing is genuinely dynamic, not a uniform grid.

Coded tools - EmployeeDirectory (identity -> authorized scope), ClientResolver (repo -> client), ScopedStandardsRetriever (reads exactly one doc and enforces isolation + access in code), and GroundingChecker (flags unsupported claims).

The key architectural point: routing is by natural-language negotiation between agents (AAOSA), while the guardrails are deterministic Python - isolation and access control cannot be talked around, and when no rule exists the system escalates instead of inventing one.

Multi-Turn Conversations & Session Memory

neuro-san's chat_context threads conversation state across turns, and sly_data carries the developer's identity and current client out-of-band. The frontend persists both in Streamlit session state, so once you say "I'm amirthap" you never have to repeat it - and cross-client questions stay blocked.

Technology Choices

Concern	Choice	Reason
Agent orchestration	neuro-san 0.6.71	Declarative HOCON agents; AAOSA routing; CodedTool interface; multi-turn context
LLM	Anthropic claude-sonnet	Strong instruction-following for routing and grounded answers
Frontend	Streamlit 1.56.0	Rapid chat UI; session state for context + sly_data
Retrieval	Local Markdown files	Reliability over sophistication for a live demo (no vector DB)
Agent config	HOCON	Comments + includes; readable hierarchical multi-agent topologies

---

Project Structure

```
Multi-Client-Standards-Concierge/
|-- backend/
|   |-- run.py                          # Backend entry point & invoke_agent()
|   |-- registries/                     # manifest, network, aaosa, llm_config
|   \-- coded_tools/standards_concierge/  # 4 coded tools + 64 synthetic docs
|-- frontend/
|   \-- run.py                          # Streamlit chat interface
|-- requirements.txt
|-- architecture.md
|-- summary.md
\-- README.md
```

What Makes This Interesting for a Hackathon

- Three genuine routing levels, not a flattened lookup - client, team, and topic are each decided by an agent.
- Same question, opposite answers across clients - a vivid, one-glance demo.
- Deterministic guardrails: isolation and access control are enforced in code, so the behaviour is reliable, not prompt-dependent.
- Grounded answers: an evaluation loop flags hallucinations; missing rules escalate instead of being invented.
- Identity-aware: a developer only ever sees the client they are authorized for.

Example Interaction

```
User:   Hi, I'm amirthap. Can I use a GPL-licensed library?
Agent:  Hi amirthap - you're on Client_Banking (Team_A). No: GPL/AGPL/LGPL
        dependencies are prohibited in any Banking service. Consider a
        permissively-licensed alternative, or escalate to the architect.

User:   Show me the Automotive client's security standards.
Agent:  You're authorized for Client_Banking only, not Automotive - I can't
        show you another client's standards.
```

Future Work

- Swap synthetic Markdown for a governed standards repository per client.
- Add an audit log of who asked what, for which client.
- Optional richer retrieval (hybrid search) once reliability is proven.
- Role tiers (e.g., architects can read all teams within their client).